

Doc. no. P0005R1

Date: 2015-10-18

Project: Programming Language C++

Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Stephan T. Lavavej <stl@microsoft.com>

Tomasz Kamiński <tomaszkam at gmail dot com>

Adopt `not_fn` from Library Fundamentals 2 for C++17

Table of Contents

- [Revision History](#)
 - [Revision 0](#)
 - [Revision 1](#)
- [Introduction](#)
- [Recommended for Immediate Adoption](#)
- [Remove Support for the Adaptable Function Protocol](#)
 - [Remove the Classic `not1` and `not2` Negators](#)
 - [Remove the typedefs that Support Adaptable Functions](#)
 - [Allow Vendors to Support Customers](#)
 - [Precedents for Immediate Removal](#)
- [Future Extensions](#)
 - [reference_wrapper for Incomplete Types](#)
 - [Negated bind Expressions](#)
- [Established Experience](#)
- [Proposed Wording](#)
- [References](#)

Revision History

Revision 0

Original version of the paper for the 2015 pre-Kona mailing.

Revision 1

Revised to follow [P0090R0](#), eliminating *weak result type* and `result_type`, and adding compatibility clauses to Annex C.

Additional future work item for `std::reference_wrapper`.

Established precedent for removing features from the standard without a period of deprecation.

Introduction

This paper recommends moving the `not_fn` function binder immediately out of Library Fundamentals 2 TS, into the C++17 Working paper. There are no parts of the Fundamentals TS that depend on this new component, and it serves a bigger purpose in the main standard, allowing the deprecation and removal of the last of the legacy function binder APIs. It further recommends removing any remaining infrastructure to support the C++98 adaptable function binder protocol, that was mostly removed at the 2014 Urbana meeting.

In the same mailing as revision 0 of this paper, Stephan Lavavej proposed removing the same typedefs, along with the *weak result type* wording that completed support for the deprecated (and removed) binders. This paper agrees with that direction and merges his wording to form a single, votable paper. Stephan's proposal remains available if the committee prefer to remove the binder components without (yet) adopting the `not_fn` replacement.

Recommendation for Immediate Adoption

The function template `not_fn` is first proposed by Tomasz Kamiński for Library Fundamentals 2, and so has not yet been published. However, there is extensive experience of this feature through the Boost library. The main reason cited for putting

this into Fundamentals 2, rather than directly into C++17, was to give users early access to the feature. However, we are already seeing new C++17 library features available in the main standard library distributions, typically guarded by an experimental C++17 build mode flag.

The benefit of adopting this library directly into the C++ Standard is that it is the necessary missing component to allow us to deprecate the legacy negator types, `unary_negate` and `binary_negate`, and the factory functions `not1` and `not2`. These are the last two library components that depend on the adaptable function protocol of embedding typedefs in functor classes to support adaption. The remaining facilities such as `bind1st` have already been removed, following their deprecation in C++11, and adopting `not_fn` would allow us to deprecate, and ultimately remove, this final legacy.

Remove Support for the Adaptable Function Protocol

Remove the Classic `not1` and `not2` Negators

Traditionally we would start a long deprecation process before removing supported library features such as `not1` and `not2`. However, this paper will argue that we should go further and actively remove them from the proposed C++17 standard, once the `not_fn` replacement is available, along with the adaptable functors have already been removed from the working paper.

The list of references at the end of this paper shows a number of other deprecated features targeted for removal in C++17, suggesting this would be an ideal time to perform cleanup, taking any hit of breaking compatibility with older code in one transition, rather than making each new version of the standard a risk.

Remove the typedefs that Support Adaptable Functions

Once the last of the classic binders are removed, then all of the support machinery created only for them should also be removed. That means removing all of the `argument_type`, `first_argument_type`, and `second_argument_type` typedefs in the standard library.

Note that the adaptable function protocol no longer functions as well when it was originally designed, due to the addition of new language features and libraries, such as lambda expressions, "diamond" functors, and more. This is not due to a lack of effort, but simply that it is not possible to have a unique set of typedefs for some of these types, such as polymorphic lambda objects. However, we do pay a cost for retaining support elsewhere in the library, due to the awkward conditionally defined member typedefs in several components, such as `std::function` wrapping a function type with *exactly* one or two parameters, or similarly for `std::reference_wrapper` for function references of *exactly* one or two arguments.

The original version of this paper recommended *retaining* the `result_type` typedefs under the mistaken impression they were part of the *INVOKE* protocol. Further research shows that link was severed back in 2007 with the adoption of [N2194](#) which specified `result_of` entirely in terms of `decltype`, and so removing the link between *bind*-expressions and `result_of` when using *INVOKE*. With the subsequent removal of the deprecated function binders, and the proposed removal of the classic negators in this paper, there is no further use of weak result types in the standard library, other than defining them. Therefore, I agree with Stephan Lavavej in paper [P0090R0](#) and recommend the removal *weak result type* and related uses of `result_type` from the standard library too.

Allow Vendors to Support Customers

Library vendors will often choose to continue supporting customers using previous library features long after the standard has removed them, especially if they support multiple versions of the standard through the same distribution. Stephan proposed adding a subsection to clause 17 reserving certain names for use by *previous* standards. This continues to reserve those names for use by the library, meaning that users are not permitted to start using these names as macros. It also allows vendors to remove these identifiers at a time of their choosing, rather than immediately on the release of the new standard. This will mitigate the previous removal of deprecated features like `auto_ptr`.

This paper adopts Stephan's wording with the minor alteration that continued support of the adaptable function typedefs is unspecified, rather than implementation defined. Stephan has a preference for his original wording, while Alisdair is concerned about mandating documentation for something outside the standard, so there should be at least some explicit LEWG guidance on which direction is preferred.

Precedents for Immediate Removal

Removing a feature directly from the standard without a period of deprecation is unusual, but not without precedent. C++11 removed exported templates and the original meaning of the keyword `auto`. C++14 removed `gets` and the implicit `const` on `constexpr` functions. The C++17 working draft has already removed support for trigraphs.

Future Extensions

reference_wrapper for Incomplete Types

Removing the `result_type` typedefs would enable support for incomplete types in `reference_wrapper`. That is not possible today due to the need to look inside the template type parameter to sniff out the presence of a weak result type, or any of the other nested typedefs. The key difference to the current proposal is that `reference_wrapper` would be actively prohibited from adding the previous (conditional) typedefs for compatibility purposes otherwise permitted by the proposed zombie-names clause. Additionally, 17.6.4.8 [res.on.functions]/2.5 still holds, making it clear that we would be required to call out this support for `reference_wrapper`. In the meantime, implementations would be free to document and extend this guarantee as a conforming extension.

Adopting this suggestion would enable code like the following, that is currently ill-formed if the template type parameters are incomplete when instantiated:

```
template<typename T, typename U>
auto my_tie(T& t, U& u) {
    return std::make_tuple(ref(t), ref(u));
}
```

Negated bind Expressions

Another feature of the Boost library that contributed the original design for `not_fn` is an expansion of the `bind` language to return a bind expression that negates its result when called as `!bind(a1, a2, ..., aN)`.

While this feature is desirable, it has not yet been through the LEWG process, and is slightly trickier to word than implement as the result of a 'bind' expression is unspecified, and the interaction works by providing an operator! overload for the unspecified bind-object type. Therefore, this is left as an extension to be proposed for the Library Fundamentals 3 TS.

Established Experience

The `not_fn` library, and its interaction with `bind`, have been popular parts of the Boost library distribution for over a decade now. The interface and specification are mature.

Proposed Wording

Insert a new section under 17.6.4.3 [reserved.names], between 17.6.4.3.1 [macro.names] and 17.6.4.3.2 [extern.names]:

17.6.4.3.x Zombie names [zombie.names]

In namespace `std`, the following names are reserved for previous standardization: `auto_ptr`, `bind1st`, `bind2nd`, `binder1st`, `binder2nd`, `mem_fun`, `mem_fun_ref`, `mem_fun_t`, `mem_fun1_t`, `mem_fun_ref_t`, `mem_fun1_ref_t`, `const_mem_fun_t`, `const_mem_fun1_t`, `const_mem_fun_ref_t`, `const_mem_fun1_ref_t`, `not1`, `not2`, `unary_negate`, `binary_negate`, `ptr_fun`, `pointer to unary function`, `pointer to binary function`, `random_shuffle`, `unary function`, and `binary function`.

It is implementation-defined whether function objects in the C++ standard library additionally provide the following typedefs: `result_type`, `argument_type`, `first_argument_type`, and `second_argument_type`.

Amend the `<function>` header synopsis in 20.0p2 [function.objects]:

```
// 20.9.9, negators:
template <class Predicate> class unary_negate;
template <class Predicate>
    constexpr unary_negate<Predicate> not1(const Predicate&);
template <class Predicate> class binary_negate;
template <class Predicate>
    constexpr binary_negate<Predicate> not2(const Predicate&);

// 20.9.9, Function template not_fn
template <class F> unspecified not_fn(F&& f);
```

Strike the `result_type`, `first_argument_type`, and `second_argument_type` typedefs from 20.8.2.4 [util.smartptr.ownerless].

20.8.2.4 Class template owner_less [util.smartptr.ownerless]

1 The class template `owner_less` allows ownership-based mixed comparisons of shared and weak pointers.

```

namespace std {
    template<class T> struct owner_less;

    template<class T> struct owner_less<shared_ptr<T> > {
        typedef bool result_type;
        typedef shared_ptr<T> first_argument_type;
        typedef shared_ptr<T> second_argument_type;
        bool operator()(shared_ptr<T> const&, shared_ptr<T> const&) const;
        bool operator()(shared_ptr<T> const&, weak_ptr<T> const&) const;
        bool operator()(weak_ptr<T> const&, shared_ptr<T> const&) const;
    };

    template<class T> struct owner_less<weak_ptr<T> > {
        typedef bool result_type;
        typedef weak_ptr<T> first_argument_type;
        typedef weak_ptr<T> second_argument_type;
        bool operator()(weak_ptr<T> const&, weak_ptr<T> const&) const;
        bool operator()(shared_ptr<T> const&, weak_ptr<T> const&) const;
        bool operator()(weak_ptr<T> const&, shared_ptr<T> const&) const;
    };
}

```

Strike the last note in 20.9p5 [function.objects]:

5 [Note: To enable adaptors and other components to manipulate function objects that take one or two arguments many of the function objects in this clause correspondingly provide typedefs argument_type and result_type for function objects that take one argument and first_argument_type, second_argument_type, and result_type for function objects that take two arguments. â€” end note]

Strike the definition of *weak result type* from 20.9.2:

20.9.2 Requirements [func.require]

1. Define *INVOKE* (*f*, *t*₁, *t*₂, ..., *t*_N) as follows:
 1. (*t*₁.*f*)(*t*₂, ..., *t*_N) when *f* is a pointer to a member function of a class τ and *t*₁ is an object of type τ or a reference to an object of type τ or a reference to an object of a type derived from τ ;
 2. ((**t*₁).*f*)(*t*₂, ..., *t*_N) when *f* is a pointer to a member function of a class τ and *t*₁ is not one of the types described in the previous item;
 3. *t*₁.*f* when *N* == 1 and *f* is a pointer to member data of a class τ and *t*₁ is an object of type τ or a reference to an object of type τ or a reference to an object of a type derived from τ ;
 4. (**t*₁).*f* when *N* == 1 and *f* is a pointer to member data of a class τ and *t*₁ is not one of the types described in the previous item;
 5. *f*(*t*₁, *t*₂, ..., *t*_N) in all other cases.
2. Define *INVOKE* (*f*, *t*₁, *t*₂, ..., *t*_N, *R*) as static_cast<void>(INVOKE (*f*, *t*₁, *t*₂, ..., *t*_N)) if *R* is cv void, otherwise INVOKE (*f*, *t*₁, *t*₂, ..., *t*_N) implicitly converted to *R*.
3. If a call wrapper (20.9.1) has a *weak result type* the type of its member type *result_type* is based on the type τ of the wrapper's target object (20.9.1):
 1. if τ is a pointer to function type, *result_type* shall be a synonym for the return type of τ ;
 2. if τ is a pointer to member function, *result_type* shall be a synonym for the return type of τ ;
 3. if τ is a class type and the qualified-id $\tau::\text{result_type}$ is valid and denotes a type (14.8.2), then *result_type* shall be a synonym for $\tau::\text{result_type}$;
 4. otherwise *result_type* shall not be defined.
4. Every call wrapper (20.9.1) shall be MoveConstructible. A *simple call wrapper* is a call wrapper that is CopyConstructible and CopyAssignable and whose copy constructor, move constructor, and assignment operator do not throw exceptions. A *forwarding call wrapper* is a call wrapper that can be called with an arbitrary argument list and delivers the arguments to the wrapped callable object as references. This forwarding step shall ensure that rvalue arguments are delivered as rvalue references and lvalue arguments are delivered as lvalue references. [Note: In a typical implementation forwarding call wrappers have an overloaded function call operator of the form

```

template<class... UnBoundArgs>
R operator()(UnBoundArgs&&... unbound_args) cv-qual;

```

â€” end note]

Simplify the definition of *reference_wrapper*:

20.9.4 Class template *reference_wrapper* [refwrap]

```

namespace std {
    template <class T> class reference_wrapper {
    public :
        // types
        typedef T type;
        typedef see below result_type; // not always defined
        typedef see below argument_type; // not always defined
        typedef see below first_argument_type; // not always defined
        typedef see below second_argument_type; // not always defined

        // construct/copy/destroy
        reference_wrapper(T&) noexcept;
        reference_wrapper(T&&) = delete; // do not bind to temporary objects
        reference_wrapper(const reference_wrapper& x) noexcept;

        // assignment
        reference_wrapper& operator=(const reference_wrapper& x) noexcept;

        // access
        operator T& () const noexcept;
        T& get() const noexcept;

        // invocation
        template <class... ArgTypes>
        result_of_t<T&(ArgTypes&&...)>
        operator() (ArgTypes&&...) const;
    };
}

```

1 reference_wrapper<T> is a CopyConstructible and CopyAssignable wrapper around a reference to an object or function of type T.

2 reference_wrapper<T> shall be a trivially copyable type (3.9).

3 reference_wrapper<T> has a weak result type (20.9.2). If T is a function type, result_type shall be a synonym for the return type of T.

4 The template specialization reference_wrapper<T> shall define a nested type named argument_type as a synonym for T1 only if the type T is any of the following:

(4.1) “a function type or a pointer to function type taking one argument of type T1

(4.2) “a pointer to member function R T0::f cv (where cv represents the member function's cv-qualifiers); the type T1 is cv T0*

(4.3) “a class type where the qualified-id T::argument_type is valid and denotes a type (14.8.2); the type T1 is T::argument_type;

5 The template instantiation reference_wrapper<T> shall define two nested types named first_argument_type and second_argument_type as synonyms for T1 and T2, respectively, only if the type T is any of the following:

(5.1) “a function type or a pointer to function type taking two arguments of types T1 and T2

(5.2) “a pointer to member function R T0::f(T2) cv (where cv represents the member function's cv-qualifiers); the type T1 is cv T0*

(5.3) “a class type where the qualified-ids T::first_argument_type and T::second_argument_type are both valid and both denote types (14.8.2); the type T1 is T::first_argument_type and the type T2 is T::second_argument_type;

Strike the result_type, argument_type, first_argument_type, and second_argument_type typedefs from 20.9.5 [arithmetic.operations].

20.9.5 Arithmetic operations [arithmetic.operations]

1 The library provides basic function object classes for all of the arithmetic operators in the language (5.6, 5.7).

```

template <class T = void> struct plus {
    constexpr T operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef T result_type;
};

```

2 operator() returns $x + y$.

```
template <class T = void> struct minus {
    constexpr T operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef T result_type;
};
```

3 operator() returns $x - y$.

```
template <class T = void> struct multiplies {
    constexpr T operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef T result_type;
};
```

4 operator() returns $x * y$.

```
template <class T = void> struct divides {
    constexpr T operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef T result_type;
};
```

5 operator() returns x / y .

```
template <class T = void> struct modulus {
    constexpr T operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef T result_type;
};
```

6 operator() returns $x \% y$.

```
template <class T = void> struct negate {
    constexpr T operator()(const T& x) const;
    typedef T argument_type;
    typedef T result_type;
};
```

7 operator() returns $-x$.

Strike the result_type, first_argument_type, and second_argument_type typedefs from **20.9.6 [comparisons]**.

20.9.6 Comparisons [comparisons]

1 The library provides basic function object classes for all of the comparison operators in the language (5.9, 5.10).

```
template <class T = void> struct equal_to {
    constexpr bool operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef bool result_type;
};
```

2 operator() returns $x == y$.

```
template <class T = void> struct not_equal_to {
    constexpr bool operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef bool result_type;
};
```

3 operator() returns $x != y$.

```
template <class T = void> struct greater {
    constexpr bool operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef bool result_type;
};
```

4 operator() returns $x > y$.

```
template <class T = void> struct less {  
    constexpr bool operator()(const T& x, const T& y) const;  
    typedef T first_argument_type;  
    typedef T second_argument_type;  
    typedef bool result_type;  
};
```

5 operator() returns $x < y$.

```
template <class T = void> struct greater_equal {  
    constexpr bool operator()(const T& x, const T& y) const;  
    typedef T first_argument_type;  
    typedef T second_argument_type;  
    typedef bool result_type;  
};
```

6 operator() returns $x \geq y$.

```
template <class T = void> struct less_equal {  
    constexpr bool operator()(const T& x, const T& y) const;  
    typedef T first_argument_type;  
    typedef T second_argument_type;  
    typedef bool result_type;  
};
```

7 operator() returns $x \leq y$.

Strike the result_type, argument_type, first_argument_type, and second_argument_type typedefs from **20.9.7 [logical.operations]**.

20.9.7 Logical operations [logical.operations]

1 The library provides basic function object classes for all of the logical operators in the language (5.14, 5.15, 5.3.1).

```
template <class T = void> struct logical_and {  
    constexpr bool operator()(const T& x, const T& y) const;  
    typedef T first_argument_type;  
    typedef T second_argument_type;  
    typedef bool result_type;  
};
```

2 operator() returns $x \&\& y$.

```
template <class T = void> struct logical_or {  
    constexpr bool operator()(const T& x, const T& y) const;  
    typedef T first_argument_type;  
    typedef T second_argument_type;  
    typedef bool result_type;  
};
```

3 operator() returns $x \|\| y$.

```
template <class T = void> struct logical_not {  
    constexpr bool operator()(const T& x) const;  
    typedef T argument_type;  
    typedef bool result_type;  
};
```

4 operator() returns $!x$.

Strike the result_type, argument_type, first_argument_type, and second_argument_type typedefs from **20.9.8 [bitwise.operations]**.

20.9.8 Bitwise operations [bitwise.operations]

1 The library provides basic function object classes for all of the bitwise operators in the language (5.11, 5.13, 5.12, 5.3.1).

```
template <class T = void> struct bit_and {  
    constexpr T operator()(const T& x, const T& y) const;  
    typedef T first_argument_type;  
    typedef T second_argument_type;  
    typedef T result_type;  
};
```

2 operator() returns $x \& y$.


```
template <class T = void> struct bit_or {
    constexpr T operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef T result_type;
};
```

3 operator() returns $x \mid y$.

```
template <class T = void> struct bit_xor {
    constexpr T operator()(const T& x, const T& y) const;
    typedef T first_argument_type;
    typedef T second_argument_type;
    typedef T result_type;
};
```

4 operator() returns $x \wedge y$.

```
template <class T = void> struct bit_not {
    constexpr bool operator()(const T& x) const;
    typedef T argument_type;
    typedef T result_type;
};
```

5 operator() returns $\sim x$.

Replace subclause **20.9.9 [negators]** with a new subclause **20.9.9 [func.not_fn]**, copying everything from the Library Fundamentals 2 TS other than the struck-out note:

20.9.9 Negators [negators]

1 Negators `not1` and `not2` take a unary and a binary predicate, respectively, and return their complements (5.3.1).

```
template <class Predicate>
class unary_negate {
public:
    constexpr explicit unary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::argument_type& x) const;
    typedef typename Predicate::argument_type argument_type;
    typedef bool result_type;
};
```

2 operator() returns $\neg \text{pred}(x)$.

```
template <class Predicate>
constexpr unary_negate<Predicate> not1(const Predicate& pred);
```

3 Returns: `unary_negate<Predicate>(pred)`.

```
template <class Predicate>
class binary_negate {
public:
    constexpr explicit binary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::first_argument_type& x,
        const typename Predicate::second_argument_type& y) const;
    typedef typename Predicate::first_argument_type first_argument_type;
    typedef typename Predicate::second_argument_type second_argument_type;
    typedef bool result_type;
};
```

4 operator() returns $\neg \text{pred}(x, y)$.

```
template <class Predicate>
constexpr binary_negate<Predicate> not2(const Predicate& pred);
```

5 Returns: `binary_negate<Predicate>(pred)`.

20.9.9 Function template `not_fn` [func.not_fn]

```
template <class F> unspecified not_fn(F&& f);
```


1 In the text that follows:

- `FD` is the type `decay_t<F>`,
- `fd` is an lvalue of type `FD` constructed from `std::forward<F>(f)`,
- `fn` is a forwarding call wrapper created as a result of `not_fn(f)`.

2 *Requires:* `is_constructible<FD, F>::value` shall be true. `fd` shall be a callable object (Â\$20.9.1).

3 *Returns:* A forwarding call wrapper `fn` such that the expression `fn(a1, a2, ..., aN)` is equivalent to `!INVOKE(fd, a1, a2, ..., aN)` (Â\$20.9.2).

4 *Throws:* Nothing unless the construction of `fd` throws an exception.

5 *Remarks:* The return type shall satisfy the requirements of `MoveConstructible`. If `FD` satisfies the requirements of `CopyConstructible`, then the return type shall satisfy the requirements of `CopyConstructible`. [*Note:* This implies that `FD` is `MoveConstructible`. â€” end note]

[*Note:* Function template `not_fn` can usually provide a better solution than using the negators `not1` and `not2` â€” end note]

Remove the weak result type from the type returns from `bind` expressions in **20.9.10.3 [func.bind.bind]**:

3 *Returns:* A forwarding call wrapper `g` with a weak result type (20.9.2). The effect of `g(u1, u2, ..., uM)` shall be `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN), result_of_t<FD cv & (V1, V2, ..., VN)>)`, where `cv` represents the cv-qualifiers of `g` and the values and types of the bound arguments `v1, v2, ..., vN` are determined as specified below. The copy constructor and move constructor of the forwarding call wrapper shall throw an exception if and only if the corresponding constructor of `FD` or of any of the types T_{iD} throws an exception.

Remove the `result_type` member from the type returned by `bind` expressions with user-specified return types in **20.9.10.3 [func.bind.bind]**:

7 *Returns:* A forwarding call wrapper `g` with a nested type `result_type` defined as a synonym for `R`. The effect of `g(u1, u2, ..., uM)` shall be `INVOKE(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN), R)`, where the values and types of the bound arguments `v1, v2, ..., vN` are determined as specified below. The copy constructor and move constructor of the forwarding call wrapper shall throw an exception if and only if the corresponding constructor of `FD` or of any of the types T_{iD} throws an exception.

Strike the unnecessary typedefs from the unspecified return-type of `mem_fn`:

20.9.11 Function template `mem_fn` [func.memfn]

```
template<class R, class T> unspecified mem_fn(R T::* pm);
```

1 *Returns:* A simple call wrapper (20.9.1) `fn` such that the expression `fn(t, a2, ..., aN)` is equivalent to `INVOKE(pm, t, a2, ..., aN)` (20.9.2). `fn` shall have a nested type `result_type` that is a synonym for the return type of `pm` when `pm` is a pointer to member function.

2 The simple call wrapper shall define two nested types named `argument_type` and `result_type` as synonyms for `cv T*` and `Ret`, respectively, when `pm` is a pointer to member function with cv-qualifier `cv` and taking no arguments, where `Ret` is `pm`'s return type.

3 The simple call wrapper shall define three nested types named `first_argument_type`, `second_argument_type`, and `result_type` as synonyms for `cv T*`, `T1`, and `Ret`, respectively, when `pm` is a pointer to member function with cv-qualifier `cv` and taking one argument of type `T1`, where `Ret` is `pm`'s return type.

4 *Throws:* Nothing.

Strike the relevant typedefs from the class definition of `function`:

20.9.12.2 Class template function [func.wrap.func]

```
namespace std {
template<class> class function; // undefined
template<class R, class... ArgTypes>
class function<R(ArgTypes...)> {
public:
    typedef R result_type;
    typedef T1 argument_type; // only if sizeof...(ArgTypes) == 1 and
```

```

// the type in ArgTypes is T1
typedef T1 first_argument_type; // only if sizeof...(ArgTypes) == 2 and
// ArgTypes contains T1 and T2
typedef T2 second_argument_type; // only if sizeof...(ArgTypes) == 2 and
// ArgTypes contains T1 and T2

// further details elided...
};

```

Strike bullet (1.3), on the definition of standard hash functors:

20.9.13 Class template hash [unord.hash]

1 The unordered associative containers defined in 23.5 use specializations of the class template hash as the default hash function. For all object types Key for which there exists a specialization hash<Key>, and for all enumeration types (7.2) Key, the instantiation hash<Key> shall:

(1.1) “satisfy the Hash requirements (17.6.3.4), with Key as the function call argument type, the DefaultConstructible requirements (Table 19), the CopyAssignable requirements (Table 23),

(1.2) “be swappable (17.6.3.2) for lvalues,

(1.3) “provide two nested types result_type and argument_type which shall be synonyms for size_t and Key, respectively,

(1.4) “satisfy the requirement that if $k_1 == k_2$ is true, $h(k_1) == h(k_2)$ is also true, where h is an object of type hash<Key> and k_1 and k_2 are objects of type Key;

(1.5) “satisfy the requirement that the expression $h(k)$, where h is an object of type hash<Key> and k is an object of type Key, shall not throw an exception unless hash<Key> is a user-defined specialization that depends on at least one user-defined type.

Strike the result_type, first_argument_type, and second_argument_type typedefs from the value_compare member-class of map in 23.4.4.1 [map.overview].

23.4.4 Class template map [map]

23.4.4.1 Class template map overview [map.overview]

```

namespace std {
    template <class Key, class T, class Compare = less<Key>,
              class Allocator = allocator<pair<const Key, T> > >
    class map {
    public:
        // types
        // details elided...

        class value_compare {
        friend class map;
        protected:
            Compare comp;
            value_compare(Compare c) : comp(c) {}
        public:
            typedef bool result_type;
            typedef value_type first_argument_type;
            typedef value_type second_argument_type;
            bool operator()(const value_type& x, const value_type& y) const {
                return comp(x.first, y.first);
            }
        };

        // 23.4.4.2, construct/copy/destroy:
        // remaining details elided...
    };
}

```

Strike the result_type, first_argument_type, and second_argument_type typedefs from the value_compare member-class of multimap in 23.4.5.1 [multimap.overview].

23.4.5 Class template multimap [multimap]

23.4.5.1 Class template multimap overview [multimap.overview]

```

namespace std {
    template <class Key, class T, class Compare = less<Key>,
              class Allocator = allocator<pair<const Key, T> > >
    class multimap {
    public:
        // types
        // details elided...

        class value_compare {
        friend class map;
        protected:
            Compare comp;
            value_compare(Compare c) : comp(c) {}
        public:
            typedef bool result_type;
            typedef value_type first_argument_type;
            typedef value_type second_argument_type;
            bool operator()(const value_type& x, const value_type& y) const {
                return comp(x.first, y.first);
            }
        };

        // 23.4.4.2, construct/copy/destroy:
        // remaining details elided...
    };
}

```

Add a new section under C.4 [diff.cpp14], between C.4.1 [diff.cpp14.lex] and C.4.2 [diff.cpp14.depr]:

C.4.x Clause 20: general utilities library [diff.cpp14.utilities]

Change: The typedefs `result_type`, `argument_type`, `first_argument_type`, and `second_argument_type` are not defined by function objects.

Rationale: Superseded by new features, including `decltype` and forwarding references.

Effect on original feature: Valid C++ 2014 code that uses these typedefs may fail to compile in this International Standard.

Change: The class templates `unary_negate` and `binary_negate` and the function templates `not1` and `not2` are not defined.

Rationale: Superseded by new features, including generic lambdas and the function template `not_fn`.

Effect on original feature: Valid C++ 2014 code that uses these class templates and function templates may fail to compile in this International Standard.

Optionally excise clause 3 from the Library Fundamentals 2 TS, as it is no longer required (section numbering per [n4481](#)).

References

- [N2194](#) `decltype` for the C++0x Standard Library, Douglas Gregor, Jaakko Järvi
- [N4076](#) A proposal to add a generalized callable negator, Tomasz Kamiński
- [N4086](#) Removing trigraphs?!, Richard Smith
- [N4168](#) Removing `auto_ptr`, Billy Baker
- [N4190](#) Removing `auto_ptr`, `random_shuffle()`, And Old `<functional>` Stuff, Stephan T. Lavavej
- [P0001R0](#) Remove Deprecated Use of the `register` Keyword, Alisdair Meredith
- [P0002R0](#) Remove Deprecated `operator++(bool)`, Alisdair Meredith
- [P0003R0](#) Remove Deprecated Exception Specifications from C++17, Alisdair Meredith
- [P0004R0](#) Remove Deprecated `iostreams` aliases, Alisdair Meredith
- [P0090R0](#) Removing `result_type`, etc., Stephan T. Lavavej